

Rethink Clinical — Test Automation Kickoff

Getting started with AIDLC · Playwright + Cucumber (BDD)

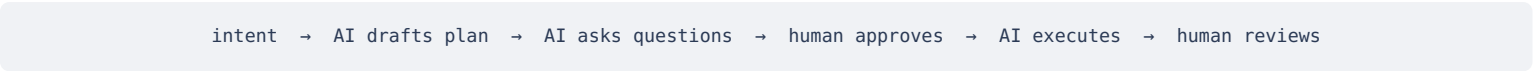
Starter guide generated from the site-crawler snapshot · dev2 environment

This guide is the **starting process** for building automated API and UI regression tests for the Rethink Clinical Angular app, using the AI-Driven Development Lifecycle (AIDLC). Follow it top to bottom. It pairs with the starter kit files: the steering files in `rules/`, the `docs/coverage-matrix.md`, and the `docs/aidlc-process.md` runbook.

Before anything else — security. The crawl file `api-endpoints.json` contains a real hashed password and API key. Do not commit the crawl folder to a shared repo and do not copy any values out of it. All credentials go into a local `.env` / CI secrets, never into source or feature files.

What AIDLC is

AIDLC runs in three phases — **Inception, Construction, Operations** — executed in short "bolts" (hours to days), with a human validation gate on every AI-proposed plan. The whole method is one repeating loop:



The AI proposes; the human approves. You never let the agent skip straight to generating code.

Step 0 — One-time setup

1. Create the project from the starter kit (files already scaffolded in `rethink-e2e/`).
2. Install dependencies and the browser:

```
npm install
npx playwright install chromium
cp .env.example .env
```

3. Fill `.env` with a **dedicated dev test account** (get one from the team) and the `AUTH_APPLICATION_KEY` from `/runtime-config.json`.
4. Load the steering files in `rules/` into your AI agent's context. These are the persistent source of truth across all three phases.

Two blockers to resolve up front. (1) How tests authenticate — replay the `POST auth/login` API for speed, plus one UI login smoke test. (2) How tests get client and program IDs — resolve them at runtime from the clients list, never hardcode an ID that could vanish.

Phase 1 — Inception (what & why)

RITUAL: MOB ELABORATION

QA, a developer, and an app owner in one session with the AI.

1. Point the AI at `rules/` and the crawl `output/` folder.
2. State the intent: "Automated API + UI regression coverage for Rethink Clinical."
3. Let the AI interrogate the crawl and ask clarifying questions live — which clients are stable test data, whether the `behaviorplans` 500 is a bug or expected, and what is out of scope.
4. Approve the coverage matrix and the ordered **units of work** in `docs/coverage-matrix.md`.

Exit gate: a signed-off coverage matrix. Nothing gets coded before this.

Phase 2 — Construction (how)

RITUAL: MOB CONSTRUCTION

One unit of work per bolt, in matrix order.

Build order: **Auth → Clients → Programs → Analyze-Data → Negative cases**. Do Auth first — nothing runs without it. Then API contract tests (fast, stable), then UI end-to-end, then network-assertion tests.

Each bolt follows the loop:

1. AI proposes the design for the unit (Page Objects, steps, fixtures). You approve.
2. AI generates feature files, step definitions, and Page Objects / API client — grounded in the real crawl data (response shapes, selectors from `raw.html` , expected XHRs).
3. You review, wire env-var auth, run, and iterate to green.
4. Update the matrix status; close the bolt; start the next unit.

Run tests as you go:

```
npm run test:smoke      # @smoke first
npm run test:api        # API contract suite
npm run test:ui         # UI end-to-end suite
```

Exit gate: `@smoke` green, then full `@api` and `@ui` green twice in a row.

Phase 3 — Operations (deploy & run)

CONTINUOUS

Get the suite running on every change and keep it healthy.

1. Generate CI config: pull creds from CI secrets, run `@smoke` on every PR, full suite nightly against dev2.
2. Publish the Cucumber HTML report and Playwright traces as build artifacts; alert on failure.
3. Feedback loop: a flake or a real bug (like `behaviorplans`) becomes a new unit of work and re-enters Inception. The lifecycle loops.

The app in one table

Read-heavy Angular SPA: 13 of 15 endpoints are GET, only login and refresh-token are POST, and there are zero HTML forms. So testing centers on navigation, data rendering, and API contracts.

Area	What to test	Type
Auth	login 200 + token, refresh, staff-role, UI login lands on /clients	@api @ui @smoke
Clients	paged envelope, item fields, list renders, open a client	@api @ui
Programs	programs, program-library, targets/objectives/mastery-criteria	@api
Analyze Data	summary cards, mastered+remaining=in-scope, chart, date chips, XHRs	@ui @network
Behavior plans	currently returns 500 — documented known defect	@api @bug

Guardrails (from the steering files)

- No secrets in the repo — ever.
- No hardcoded client or program IDs — resolve from the clients list at runtime.
- No fixed `waitForTimeout` — wait on network or element state (Angular is async-heavy).
- Ignore third-party noise (fonts).
- Every test traces to a coverage-matrix row, and a human reviews every generated test — AI-written tests can pass while asserting the wrong thing.

Your next action

Open `docs/coverage-matrix.md`, run the Inception session (Mob Elaboration) with your team, prune the matrix, then start the first Construction bolt on **Unit 1 — Authentication**. The step definitions and API client for that unit are already scaffolded to build on.

AIDLC (AI-Driven Development Lifecycle) is AWS's three-phase method (Inception, Construction, Operations), run in bolts with human approval gates. This kickoff guide adapts it to test automation for Rethink Clinical.